

Topic 2.4

2.4: typedef

The *typedef* construct lets us associate a name for often-used programmer defined types. This is both convenient and useful from a software engineering standpoint, as it allows programmers to focus on the abstract concept without the need to focus on the physical implementation of a type.

The form of a *typedef* is

```
typedef  type-definition  type-name
```

Here are some examples:

```
typedef float PRESSURE; // Representational type of
                        // a pressure is a float.

typedef float TEMPERATURE;

typedef struct {
    char name[40];
    int age;
} PERSON;

PRESSURE atmospheric_pressure;
PRESSURE bicycle_tire_pressure;
PERSON me; /* No struct required! */
PERSON my_supervisor;
```

This is useful for producing code that is more self-documenting and more portable. It does *not* prevent a wide class of errors though:

```
PRESSURE p;
TEMPERATURE t;

p = t; /* Doesn't make sense, but legal. */
```

In general, it is best to think in terms of the abstract type as long as possible, and only deal with the representational types where absolutely necessary.

Hungarian Notation

It has become a common practice to use the first character of a variable name to indicate the underlying type of the variable. There have been many different schemes in this regard, with the most widespread scheme referred to as *Hungarian notation*. The common prefix letters are defined as follows:

Hungarian Notation

Prefix	Meaning
b	boolean
c	character
i	integer
l	long
u	unsigned
p	pointer
a	array
f	bit flags
h	file handle
z	zero-terminated character array (string)
s	structure

Using the above scheme, we have example variable declarations:

```
char    cMiddleInitial;  
int     bPrinterReady;  
int     aiClassScores[100];  
char    *pzGreeting = "Hello";  
EMPLOYEE asStaff[100];
```

I have never really enjoyed the use of these naming schemes, as they tend to make you focus on the details more quickly, rather than allowing you to think at the more abstract type level longer. This has historically led to poorer code. Also, if the underlying type of a variable changes, you must change all occurrences of its name.

However, this notation or similar schemes seems ubiquitous today, especially in using MFC or OWL for C++ code development. Therefore, it is probably a good idea to become familiar with it.

Topic 2.5 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-2-dot-5>)